UNIVERSIDAD DE MÁLAGA

Departamento de Ingeniería de Sistemas y Automática

Sistemas de Tiempo Real

Dpto. de Ingeniería de Sistemas y Automática



Dr. D. Cipriano Galindo Andrades

UNIVERSIDAD DE MÁLAGA

Tema 5 – Interrupciones

5.1. Introducción

Definición. Mecanismo básico. Inconvenientes del polling. Antirrebotes.

5.2. Interrupciones Simples

Componentes. Mecanismo. Ejemplo. Borrado de interrupciones.

5.3. Interrupciones Múltiples

Anidamiento. Enmascaramiento. Jerarquización.

5.4. Programación de Interrupciones Múltiples en C

5.5. Controlador de Interrupciones Múltiples i8259 y en Atmega328p

5.6. Resumen

Departamento de Ingeniería de Sistemas y Automática

Tema 5 – Interrupciones

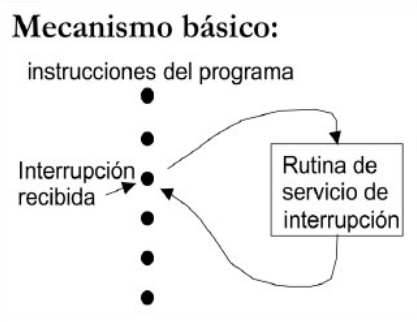
5.1. Introducción

Podemos ver el comportamiento en el osciloscopio encendiendo un led al comenzar la rutina y apagarlo al salir

Definición: Una interrupción es una indicación a un microprocesador (CPU) de la ocurrencia de un evento externo (en algunos casos se pueden disparar también por software ejecutado en la CPU).

Dado que un Sistema en Tiempo Real tiene que responder a estímulos externos en tiempo finito y especificado (por definición, ver tema 1), las interrupciones son uno de sus componentes básicos.

- Permiten responder en el menor tiempo posible: **latencia de interrupciones** -



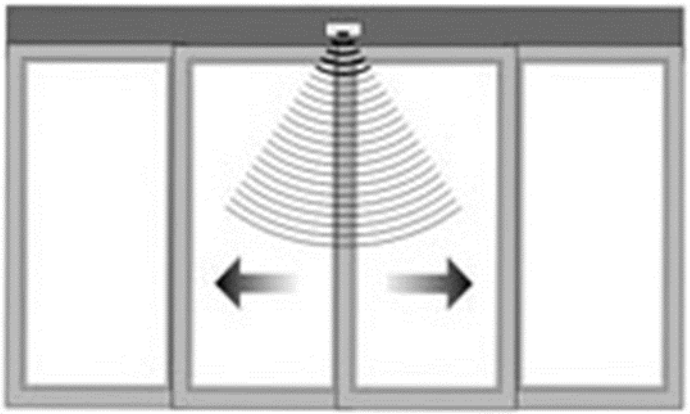


Tema 5 – Interrupciones

5.1. Introducción

Hay S.O.s que usan la rutina de interrupción sólo para activar algún indicador (*deferred intrs.*). Más adelante lanzan la verdadera ISR: así descongestionan el sistema de respuesta a señales hardware de interrupción y evitan la sobrecarga de tiempo de servicio, pero aumentan la latencia!

Ejemplo

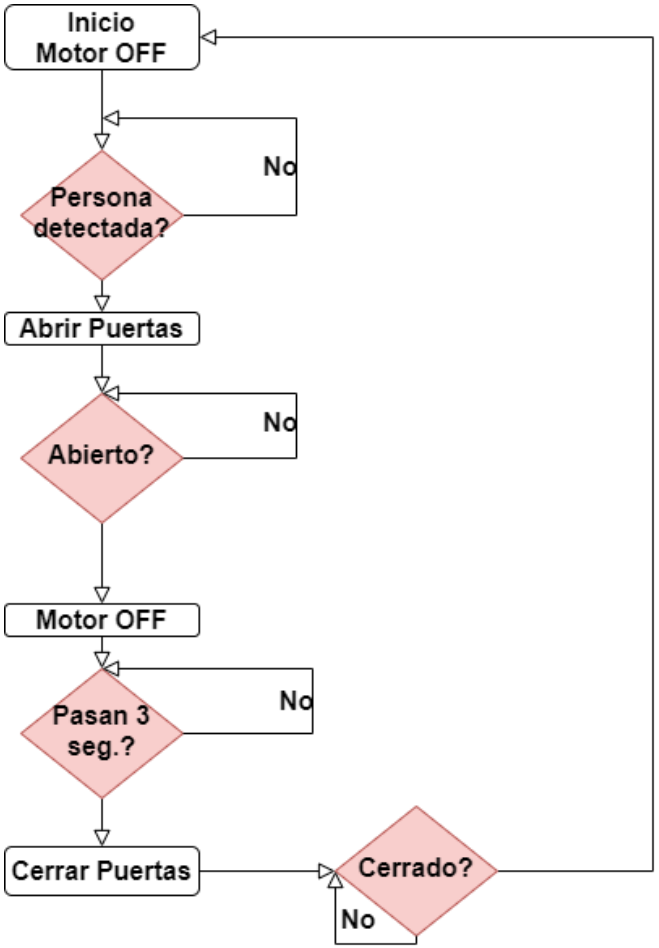


1.- El sensor detecta la llegada de una persona
2.- El motor actúa para abrir las puertas.
3.- Cuando se termina de abrir (se detecta con un final de carrera de apertura) detiene el motor.
4.- Se espera 3 segundos.
5.- El motor actúa para cerrar las puertas.
6.- Cuando se termina de cerrar (se detecta con un final de carrera de cierre), se detiene el motor

Tema 5 – Interrupciones

5.1. Introducción

Implementación con muestreo (polling no temporizado)



-Cuatro esperas de tiempo; cada bucle debe ser lo suficientemente corto para que no se produzcan "problemas", p.ej. no detecte a una persona, pasen más de 3 segundos, se queme el motor…

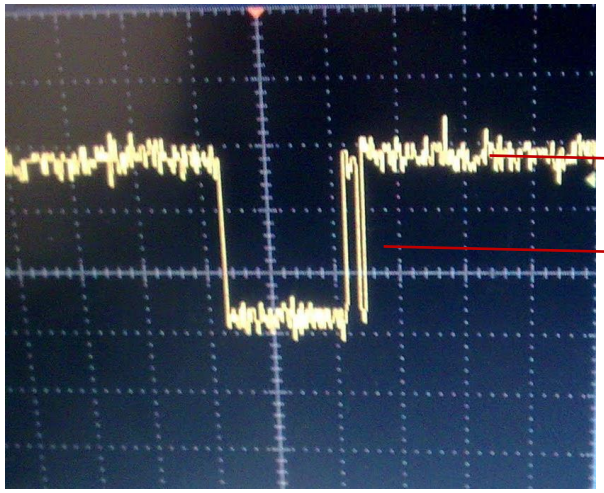
-La CPU no se puede usar para ninguna otra tarea durante cada bucle.

-Se supone que los interruptores (en los finales de carrera) no tienen rebotes

Tema 5 – Interrupciones

5.1. Introducción

Los rebotes se producen por la interacción de sistemas mecánicos y electrónicos en la generación de señales digitales. Ejemplo de rebotes en un pulsador:



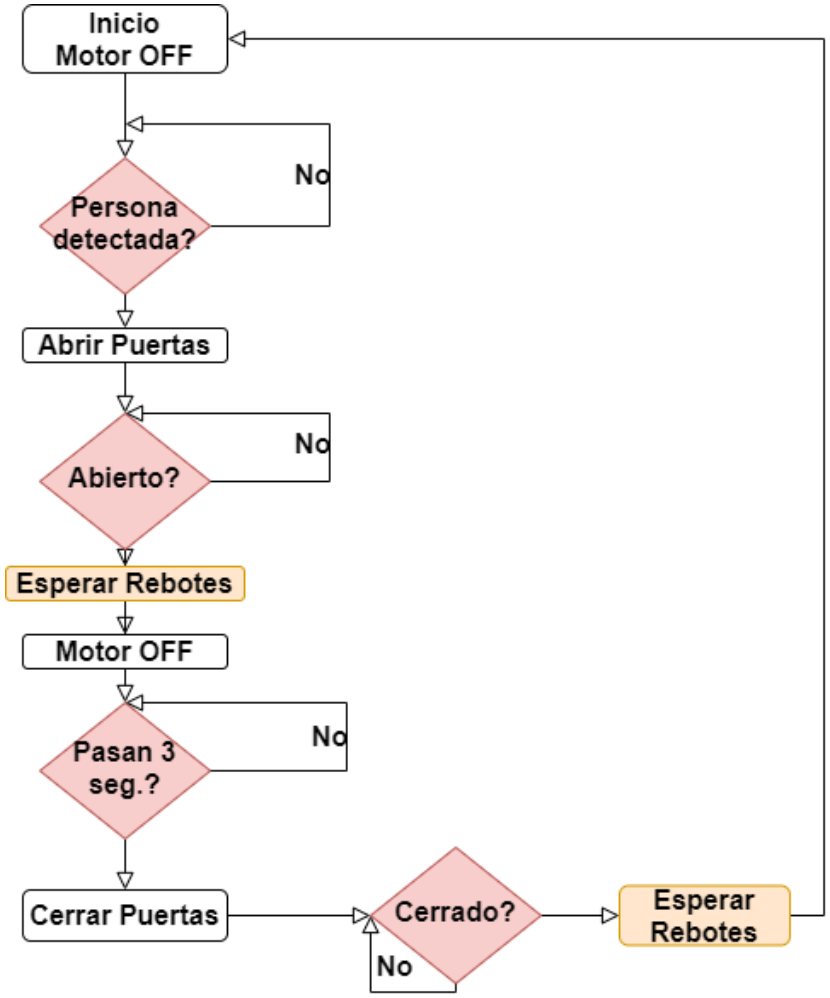
Un rebote hace como que han habido 2 interrupciones en vez de 1.
SOL: poner esperas para rebotes

Existen circuitos específicos útiles para la eliminación de rebotes. Se elimina así la necesidad de la espera de rebotes en el código.
El resultado es una señal limpia que dura tanto como lo que dure la interrupción original.

Tema 5 – Interrupciones

5.1. Introducción

Solución (mala, mediante SW) para evitar rebotes.

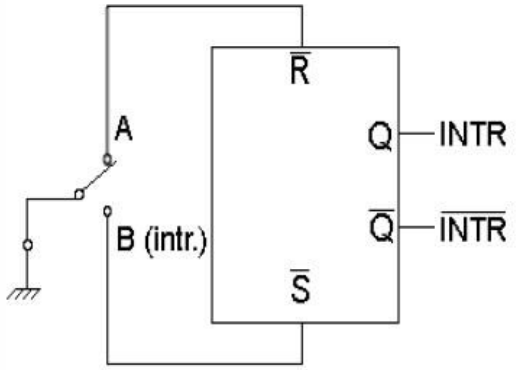


Tema 5 – Interrupciones

5.1. Introducción

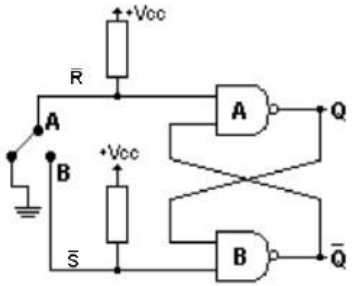
Solución a los rebotes mediante circuitería (interruptores)

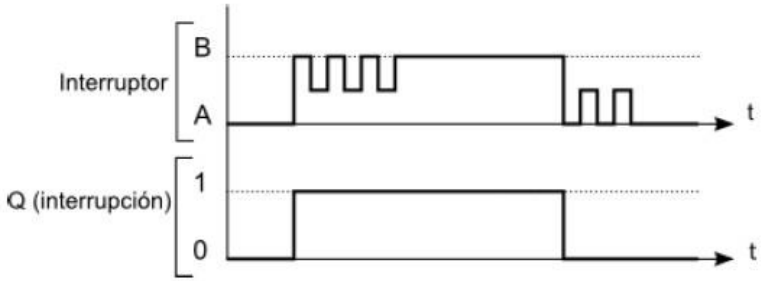
Con biestables (las entradas R y S son 1 cuando están al aire: existencia de rebotes):



Interruptor	R	S	Q	~Q
A	0	1	0	1
B	1	0	1	0
Aire (rebote)	1	1	Q anterior	~Q anterior





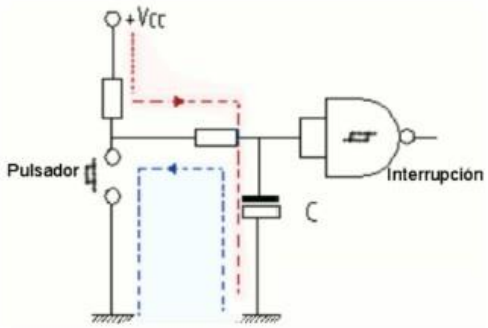


Tema 5 – Interrupciones

5.1. Introducción

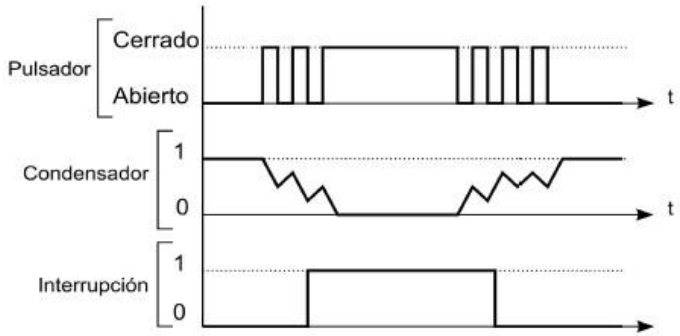
Solución a los rebotes mediante circuitería (pulsadores)

Con un condensador (carga por el camino rojo y descarga por el azul). A la salida se utiliza una puerta Schmitt para umbralizar.



Pulsador	Condensador	Interrupción
Abierto	Cargando / Cargado	0
Cerrado	Descargando / Descargado	1



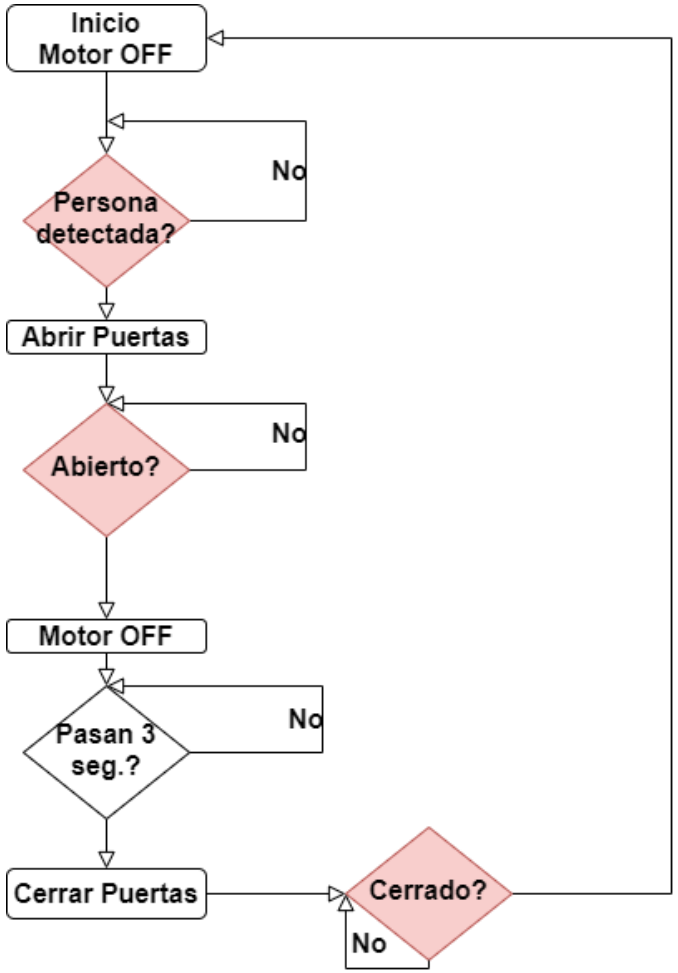


Tema 5 – Interrupciones

5.1. Introducción

Polling usando antirrebotes. (volvemos al diagrama original)

Temporización inexacta: tenemos 3 pollings no temporizados (bucles) en el sistema.



Departamento de Ingeniería de Sistemas y Automática

Tema 5 – Interrupciones

5.2. Interrupciones Simples

Solución: interrupciones simples

-Una línea de entrada de petición de interrupción (INTR), activa por nivel o por flanco.

-Un bit interno de inhibición de interrupciones (I-bit), para deshabilitar la entrada INTR.

-La petición de interrupción se traduce en una llamada a una rutina servidora de interrupción (ISR), cuya dirección en memoria es calculada por la CPU, quizás configurada por el usuario.

-Si no se puede servir la petición, se puede almacenar para cuando pueda (sólo guarda una: se pueden perder si no se gestionan suficientemente rápido).

Tema 5 – Interrupciones

5.2. Interrupciones Simples

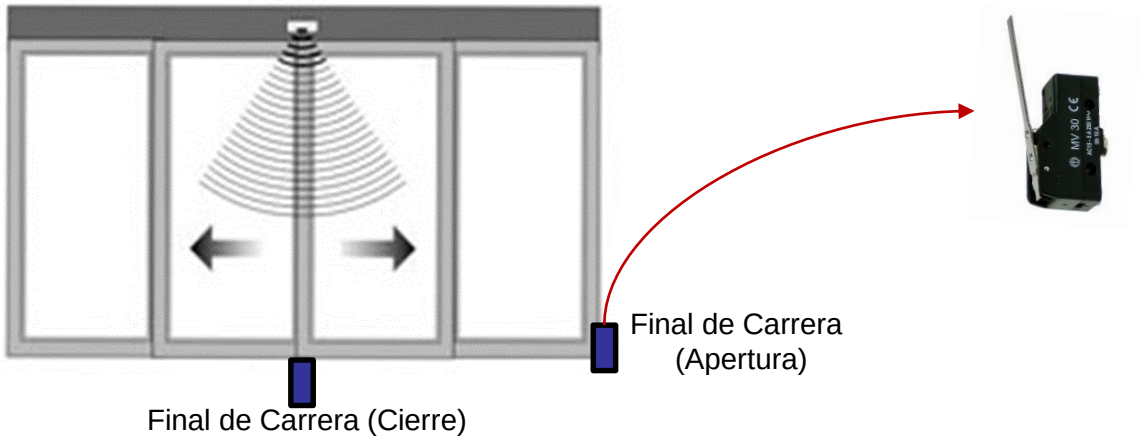
Procesamiento general de la interrupción por la CPU (si el I-bit está habilitado):

1.- La CPU termina de ejecutar la instrucción actual (si no es de repetición).
2.- Guarda la dirección de retorno al programa en la pila.
3.- Guarda el estado de la CPU (flags) en la pila.
4.- Deshabilita las futuras interrupciones deshabilitando el I-bit.
5.- Carga en el contador del programa la dirección de comienzo de la ISR, con lo que ésta se ejecuta.
... (se ejecuta la ISR; debería guardar los registros de la CPU que use y recuperarlos al terminar) ...
6.- La ISR retorna mediante una instrucción especial de fin de interrupción (RETI).
7.- RETI recupera el estado desde la pila (flags).
8.- RETI recupera de la pila la dirección de retorno y por tanto el programa interrumpido continúa.

Tema 5 – Interrupciones

5.2. Interrupciones Simples

Ejemplo implementado con Interrupciones Simples:



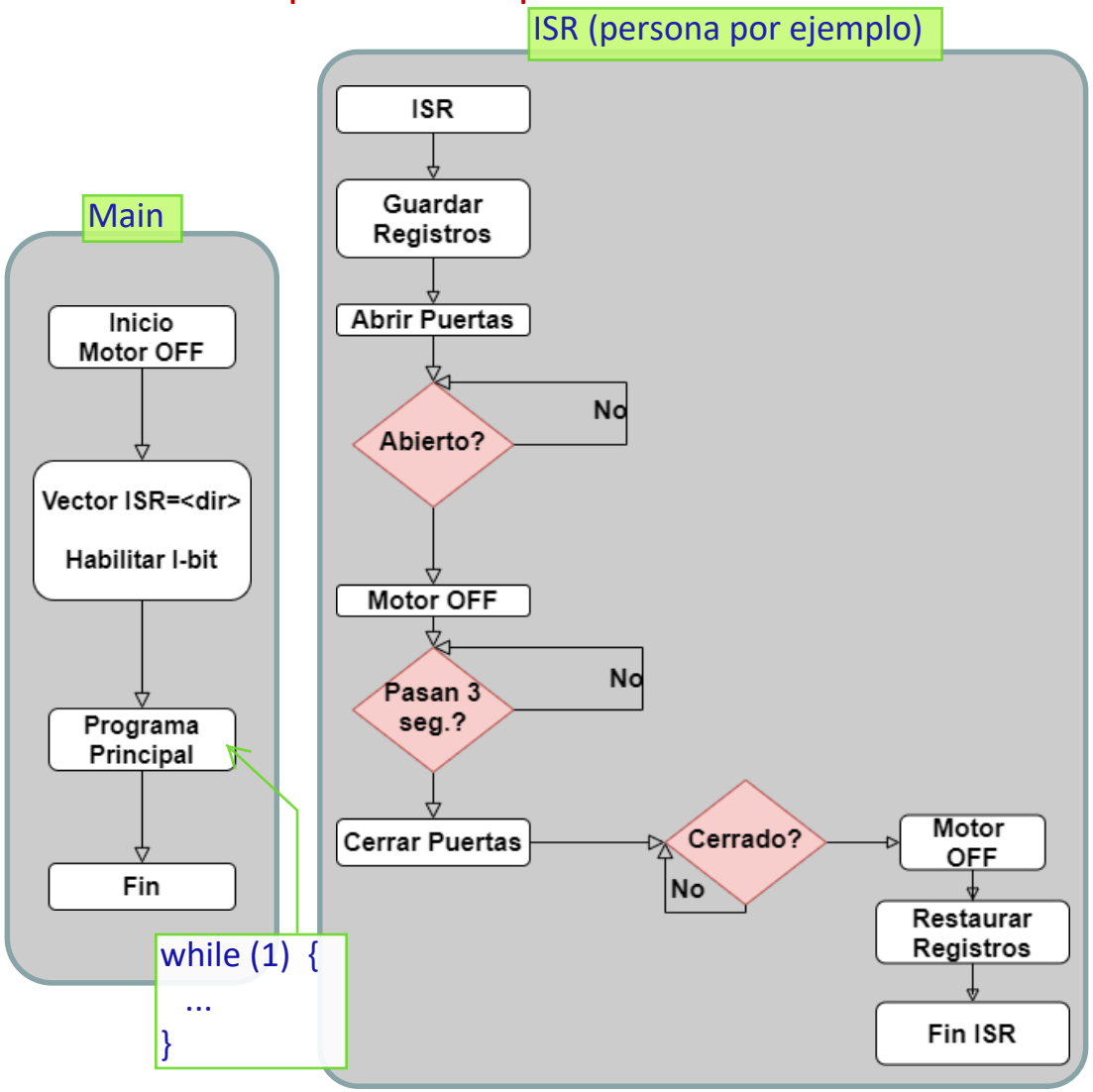
Los Finales de Carrera cambian de 1 a 0 cuando se activan y de 0 a 1 cuando se desactivan.

La CPU detecta interrupciones por flanco de bajada (de 1 a 0).

Como la llegada de interrupciones no se puede prever, estos sistemas deben depurarse mediante señales eléctricas (osciloscopios).

Tema 5 – Interrupciones

5.2. Interrupciones Simples



Se ha solucionado el bucle de la detección de personas. Quedan aun dos eventos (fin de apertura o cierre) que pueden ser imprevisibles: el motor puede tardar más o menos en función del desgaste. Solución: Interrupciones múltiples

Problemas si la señal de interrupción es por nivel en lugar de por flanco. Solución: borrado de interrupciones.

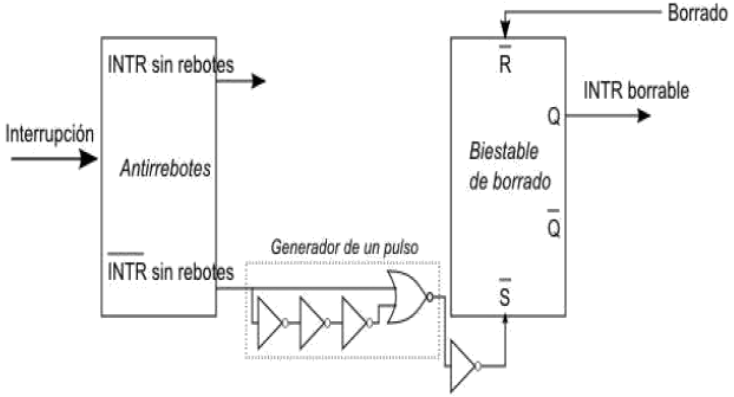
Tema 5 – Interrupciones

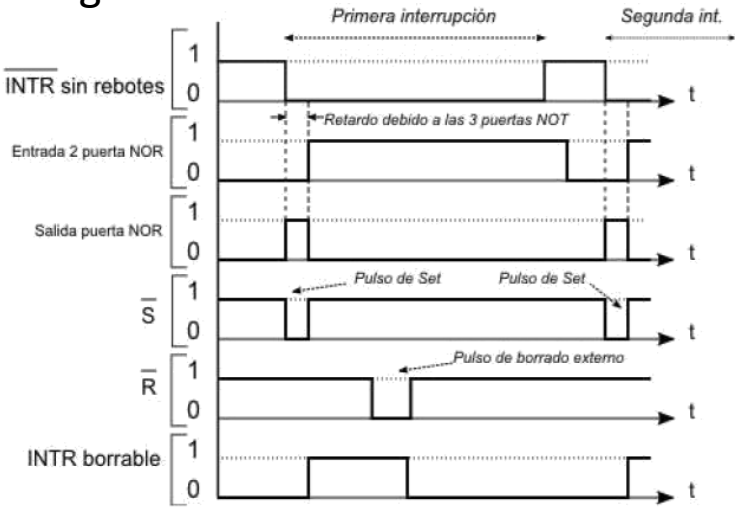
5.2. Interrupciones Simples

Solución a la repetición de la misma interrupción: borrado de interrupciones

Si INTR es activa por nivel, es necesario eliminar la señal de interrupción una vez que ésta ha sido procesada por la CPU, para no volver a procesarla una vez que la ISR termine.

-Circuito de borrado de interrupciones. Ejemplo con multivibrador monoestable/ one-shot hecho con puertas lógicas:



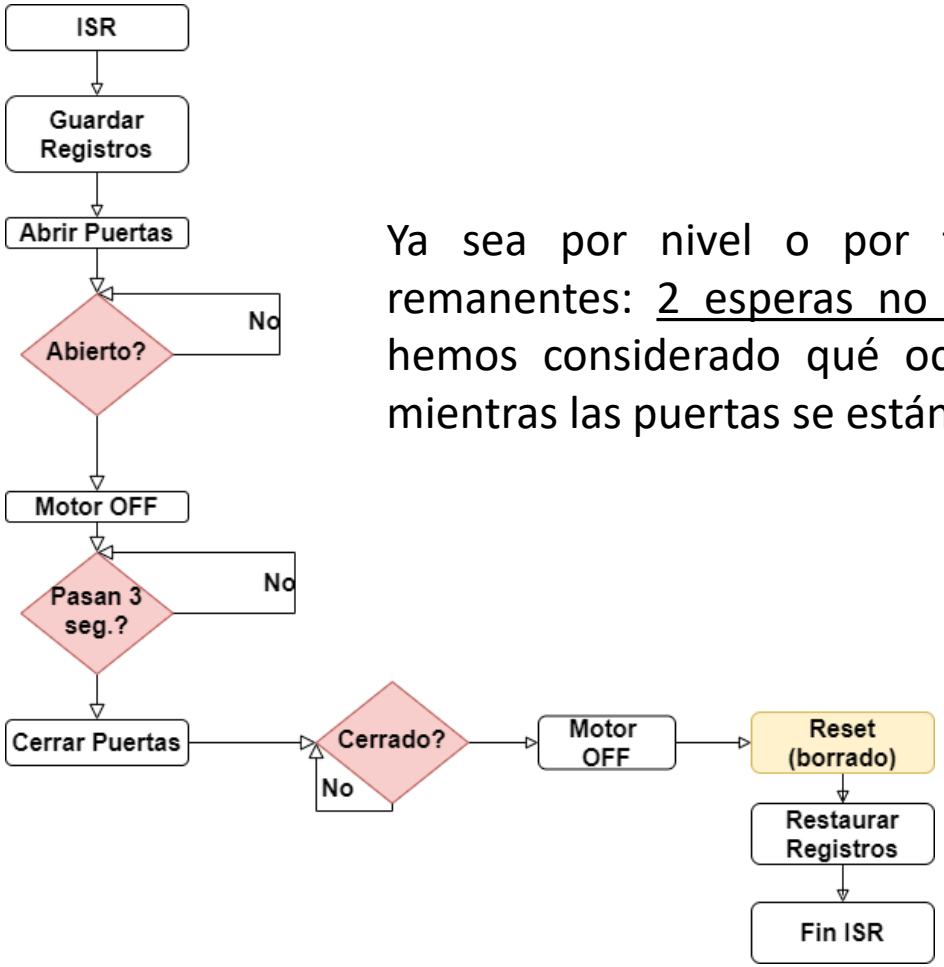


La señal "Borrado" permite borrar la interrupción ("INTR borrable") externamente.

Se memoriza la interrupción, aún cuando ya terminó la señal de interrupción (INTR sin rebotes).

Tema 5 – Interrupciones

5.2. Interrupciones Simples

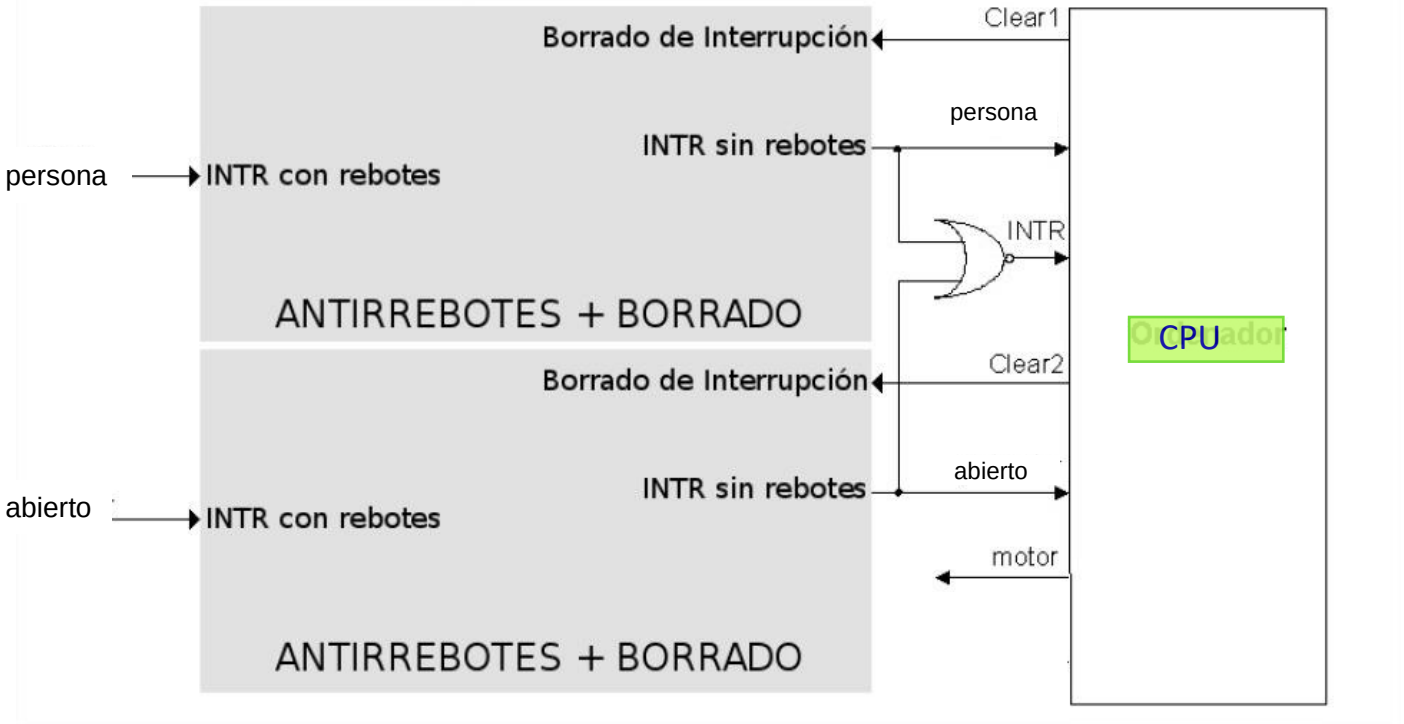


Ya sea por nivel o por flanco quedan problemas remanentes: 2 esperas no temporizadas. Además no hemos considerado qué ocurre si llega una persona mientras las puertas se están cerrando….

Tema 5 – Interrupciones

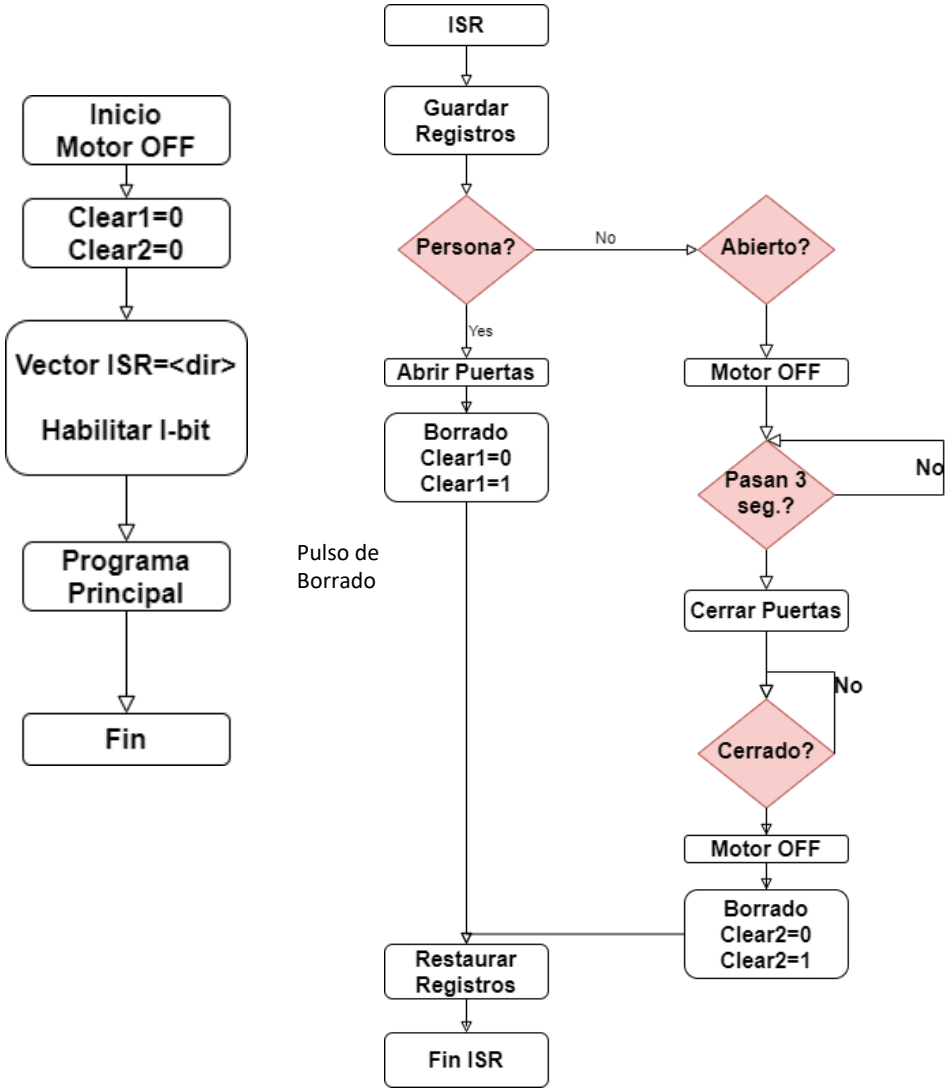
5.3. Interrupciones Múltiples

Primera Aproximación:



Tema 5 – Interrupciones

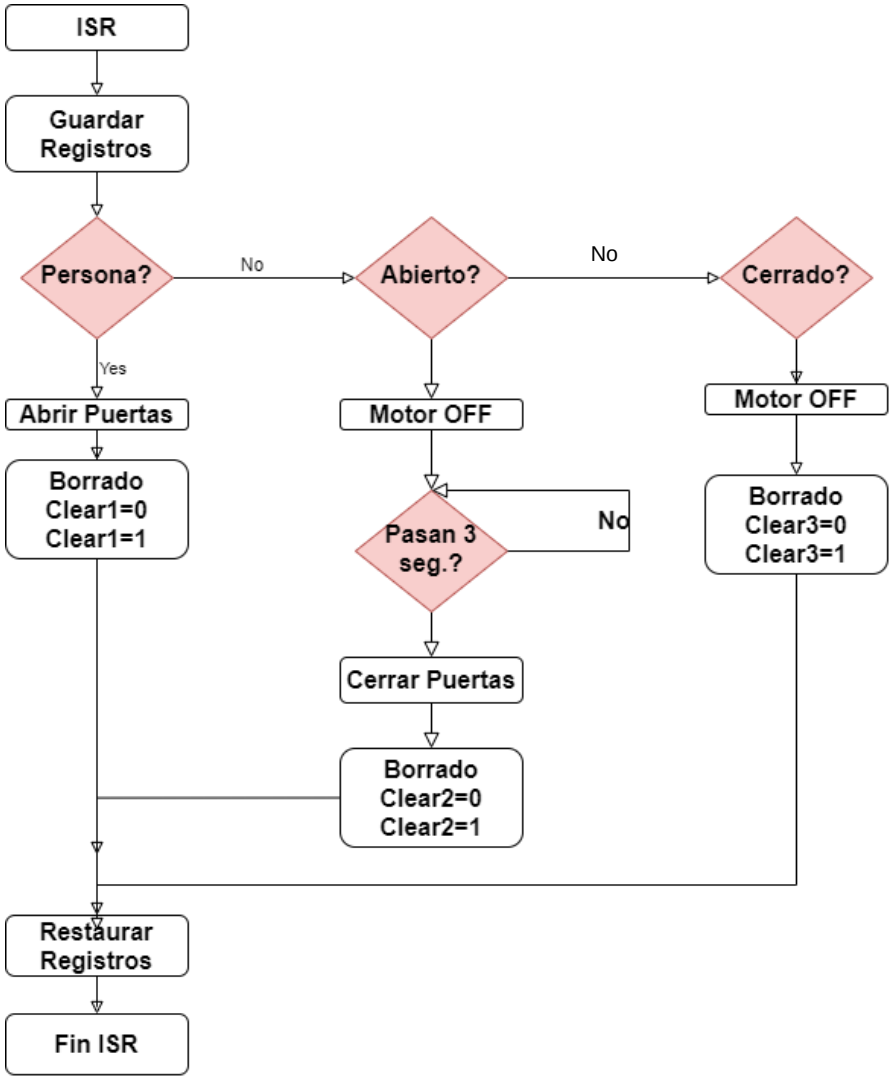
5.3. Interrupciones Múltiples



Solucionados rebotes, borrado y múltiples interrupciones (2).

Tema 5 – Interrupciones

5.3. Interrupciones Múltiples



De hecho se podría extender para una tercera interrupción… pero aparece un problema nuevo si quiero manejar prioridades.

Tema 5 – Interrupciones

5.3. Interrupciones Múltiples

Dos tipos de prioridades:

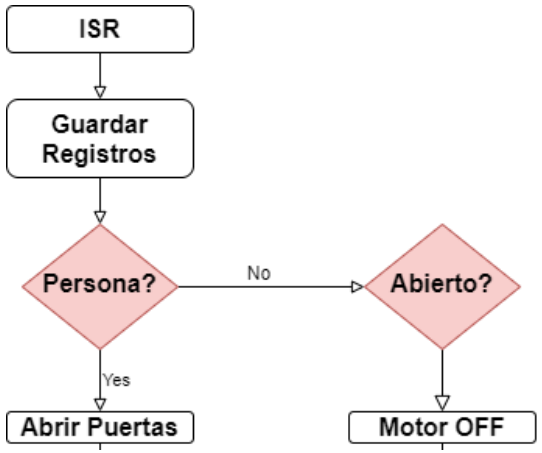
1.- Tipo a) en la ejecución de ISRs (interrupción de una ISR) y Se define en código (SW)

2.- Tipo b) en la llegada simultánea de señales de interrupción. Se define por HW

En el ejemplo:

Prioridades a) prio(principal) < prio(persona) = prio(abierto).

Prioridades b) prio(principal) < prio(abierto) < prio(persona).



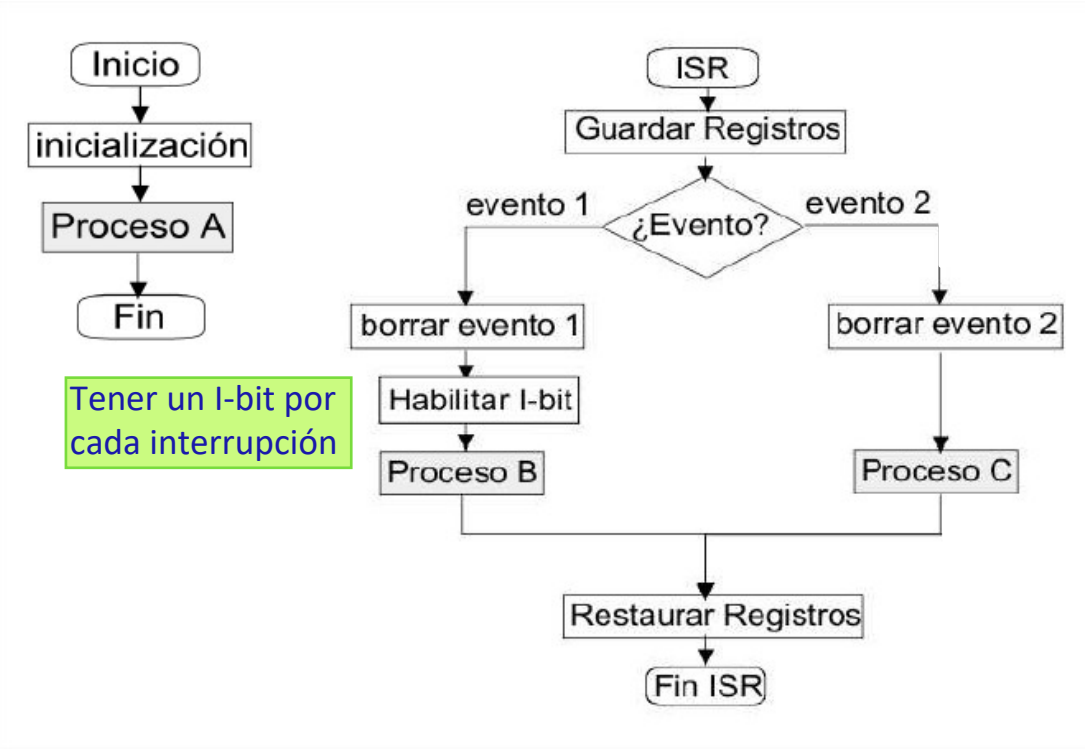
…

Tema 5 – Interrupciones

5.3. Interrupciones Múltiples

Priorización tipo (a) mediante anidamiento.

Supongamos 3 tareas: A, B, C, y que prio(A) < prio(B) < prio(C)

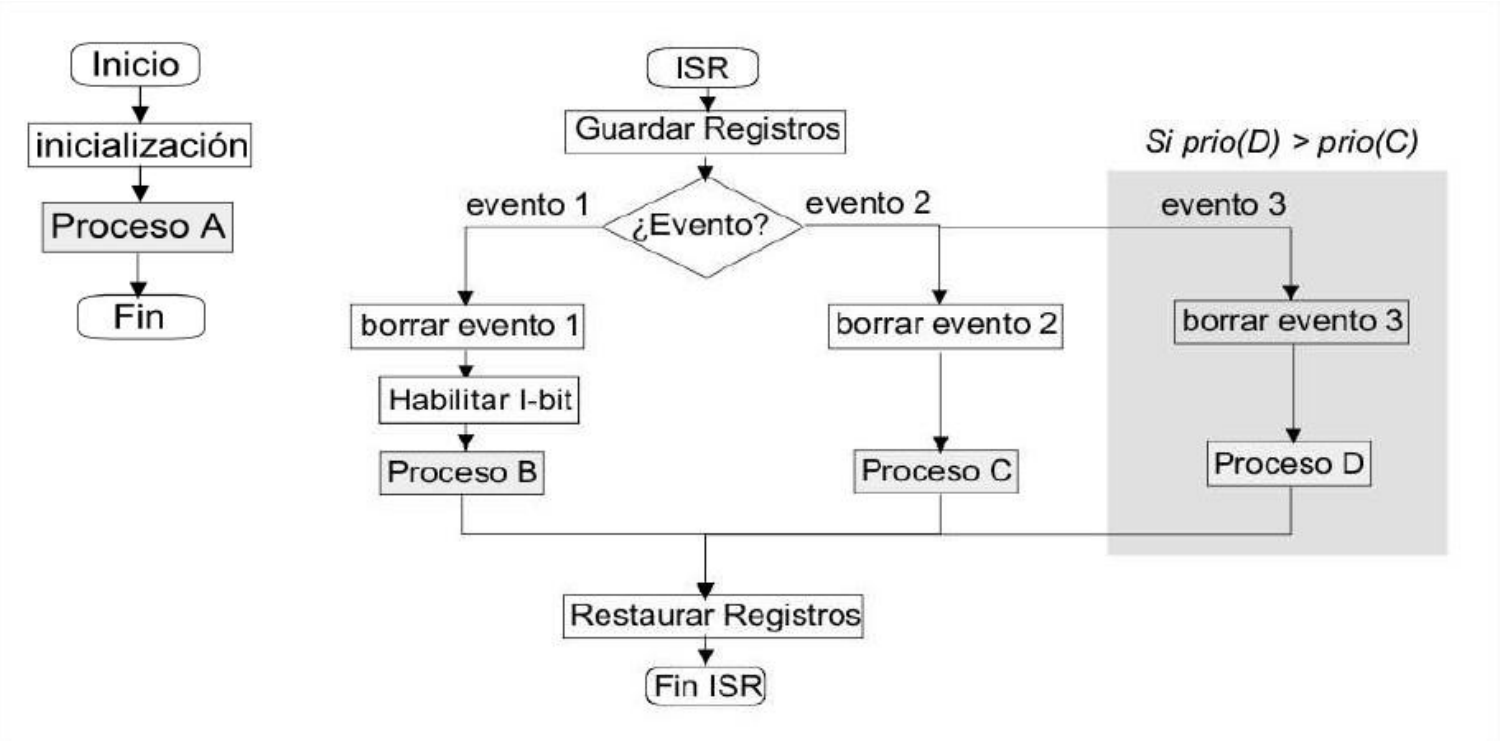


Tema 5 – Interrupciones

5.3. Interrupciones Múltiples

Priorización tipo (a) mediante anidamiento.

Supongamos 4 tareas: A, B, C, D y que prio(A) < prio(B) < prio(C) < prio (D)

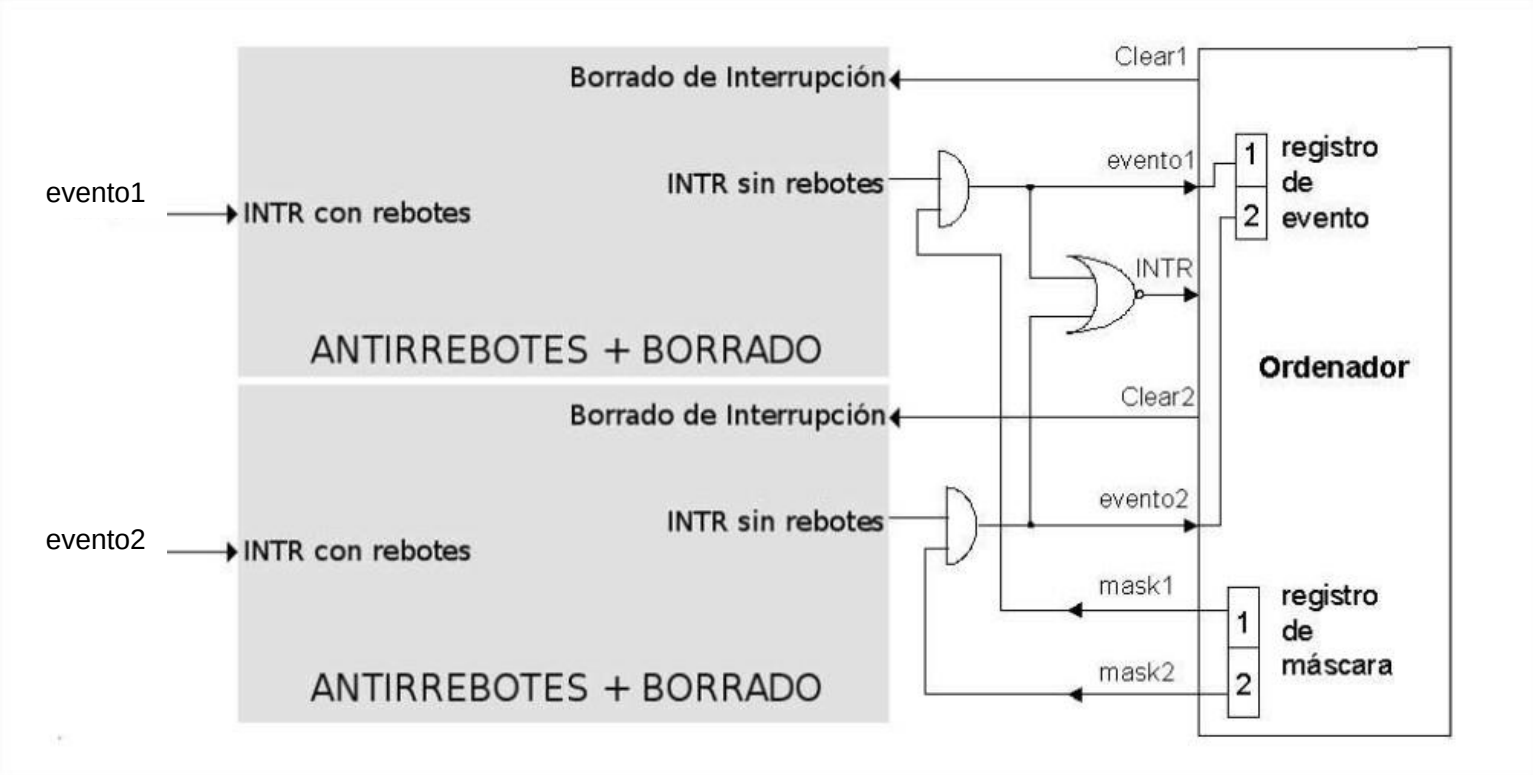


No soluciona en general el problema: sólo funciona con dos interrupciones anidadas como máximo.

Tema 5 – Interrupciones

5.3. Interrupciones Múltiples

Priorización tipo (a) mediante enmascaramiento individual.

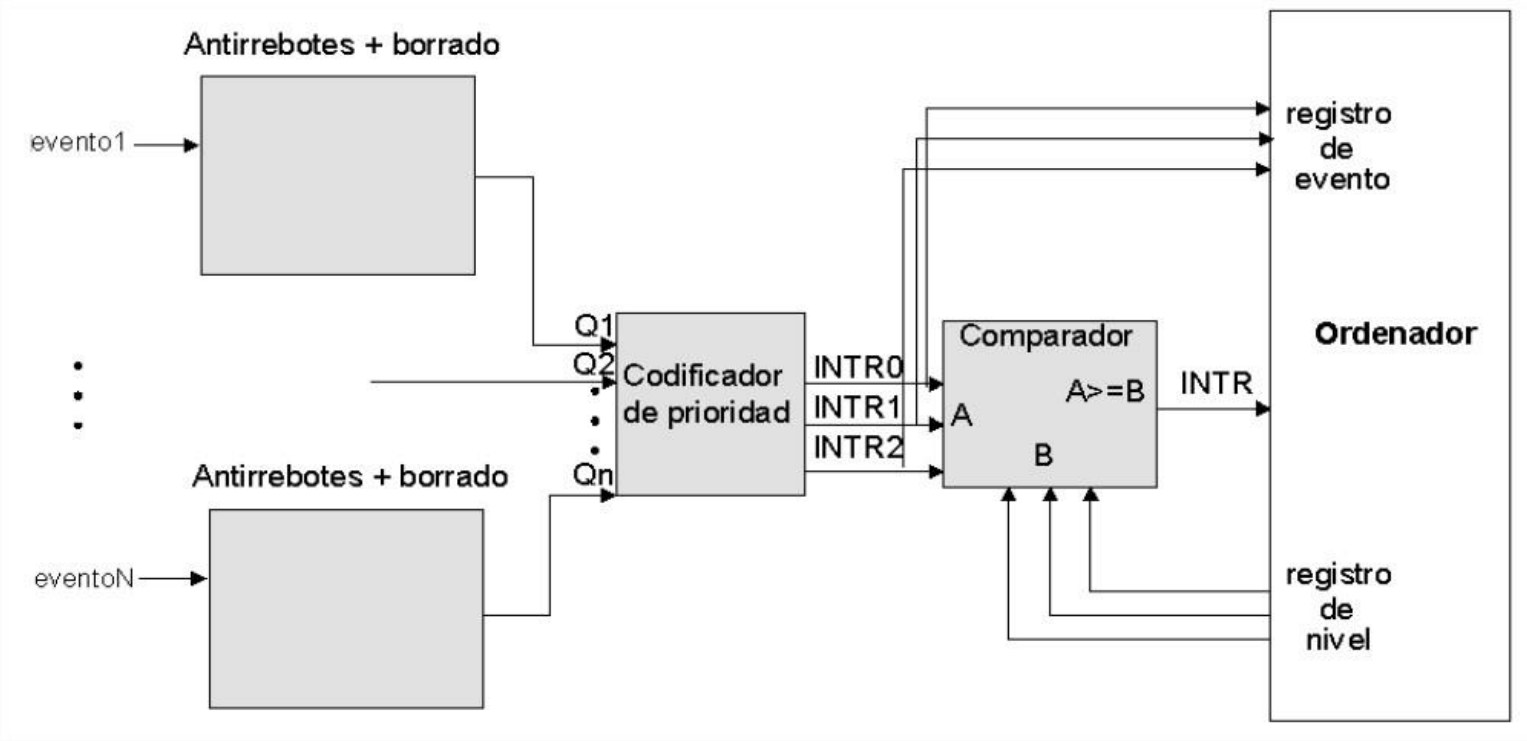


Soluciona la priorización de tipo a, pero no la priorización respecto a la llegada simultánea de señales de interrupción (tipo b)

Tema 5 – Interrupciones

5.3. Interrupciones Múltiples

Solución conjunta a la priorización tipo a y b: Jerarquización



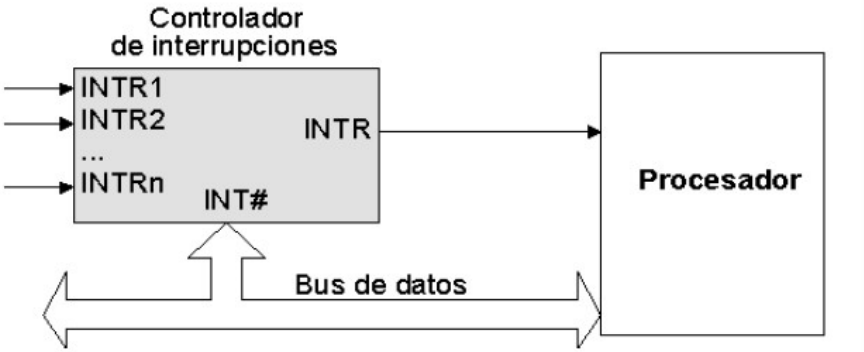
Es una variante equivalente al enmascaramiento, pero también resuelve la prioridades de tipo b (llegada simultánea de señales eléctricas de interrupción).

Tema 5 – Interrupciones

5.3. Interrupciones Múltiples

Solución integral a los sistemas con interrupciones múltiples: vectorización

Se necesita un controlador de interrupciones externo a la CPU que se ocupa del borrado, enmascaramiento y jerarquización: por lo tanto soluciona TODOS los problemas de las interrupciones múltiples.



Hay una tabla de direcciones de ISRs en memoria, indexada por el vector de interrupción (INT#), que puede ser generado por el controlador o por el dispositivo que interrumpe. Es habitual una tabla a partir de la dirección 0 de memoria, con tantos bytes por ISR como hagan falta para almacenar la dirección de comienzo de una ISR.

En nuestra CPU genérica supondremos una tabla en la dirección 0x0000 con 2 bytes por vector de ISR.

Tema 5 – Interrupciones

5.4. Programación con Interrupciones Múltiples en C

El estándar ANSI C (C90) no dispone de facilidades para gestionar interrupciones. Soluciones: añadir librerías propias de una CPU concreta: ampliaremos nuestra librería *cpu.h*:

```
//[... (cont. del tema 4) ...]
/* Una dirección de memoria */
typedef unsigned address;

/* Devuelve un puntero que apunta a una zona de memoria física que
contiene a su vez una dirección de memoria física. Por ejemplo:
devuelve un puntero a una tabla de interrupciones situada en la
posición física d */

#define ADDRESSA(d) ((address *)(d))

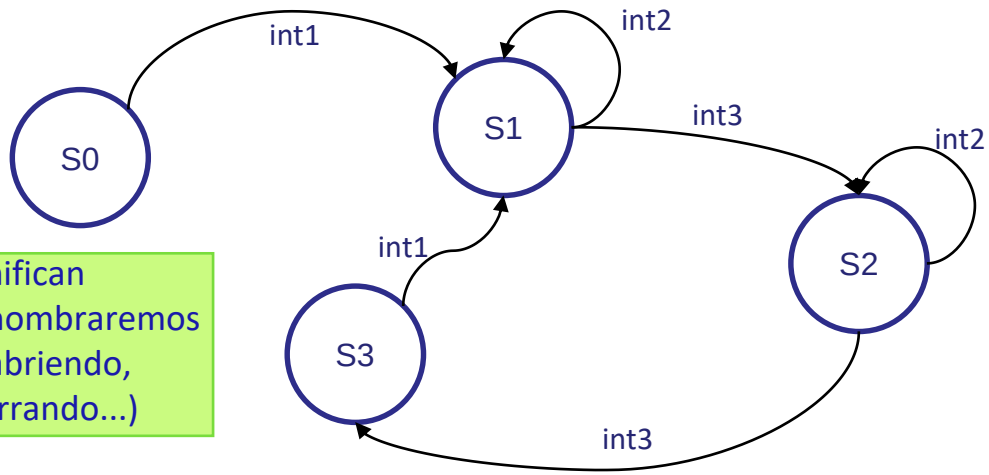
void EnableI(void);/* Habilita el I-bit */
void DisableI(void);/* Deshabilita el I-bit */
void SaveRegs(void);/* Guarda los registros de la CPU */
void RestoreRegs(void);/* Recupera los registros de la CPU */
void RetI(void);/* Recupera los flags de la pila y retorna */
```

Tema 5 – Interrupciones

5.4. Programación con Interrupciones Múltiples en C

Cada ISR será una rutina con prototipo **void *nombreISR*(void)**, cuya dirección habrá que almacenar en la tabla de interrupciones al arrancar. Pueden compartir información a través de variables globales (para enteros: *volatile sig_atomic_t*).

El programa completo será guiado por interrupciones, con lo que necesitará ser diseñado previamente. Una forma adecuada es mediante máquinas de estado finitas de Mealy: grafos en los que los nodos representan estados de espera del sistema y los arcos transiciones entre estados debidas a interrupciones:



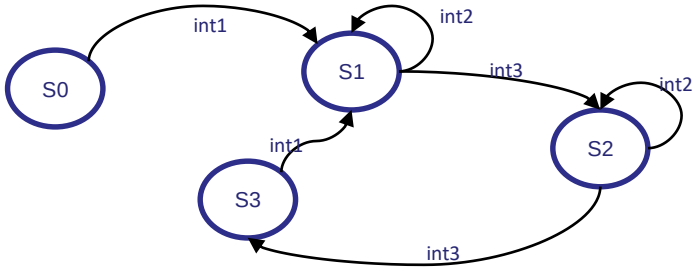
Los nodos significan esperas y los nombraremos en gerundio (abriendo, esperando, cerrando...)

Cada arco es una interrupción (podremos incluir condiciones)

Tema 5 – Interrupciones

5.4. Programación con Interrupciones Múltiples en C

Cada estado representa una espera del sistema y no llevan asociado código. De un estado se pasa a otros a través de una interrupción (y posiblemente una condición), ejecutándose el código correspondiente.



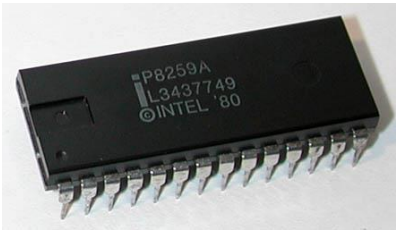
El código de las ISRs debe ser lo más simple y rápido posible para miniminizar el efecto de retardos en la ejecución de otras ISRs o en el programa principal. No deben incluir esperas activas (polling).

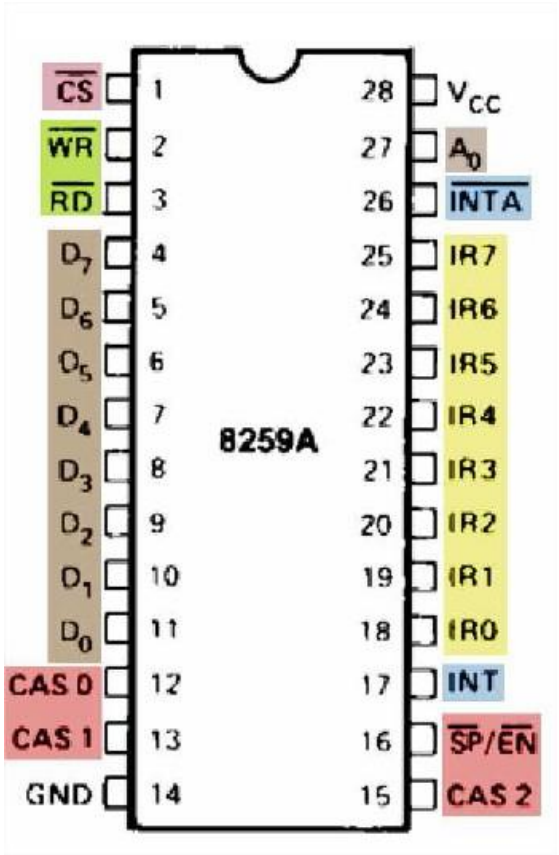
Una misma interrupción puede ser considerada en diferentes estados. Traducción directa del grafo de Mealy a código C. No hay una solución única.

Ejercicio: Diagrama de Mealy del ejemplo de la puerta automática

Tema 5 – Interrupciones

5.5. Controlador de Interrupciones Múltiples i8259





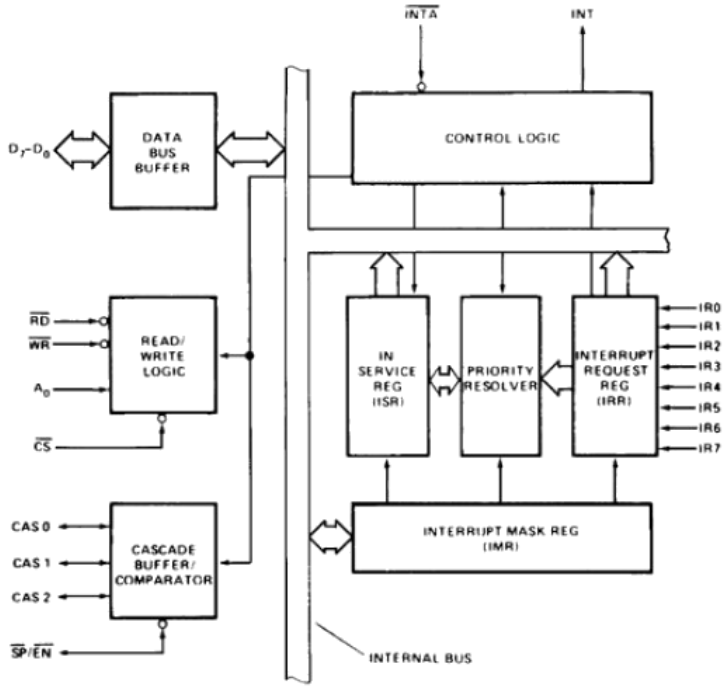
Controla hasta 8 líneas de interrupción (conectable en cascada: hasta 64 líneas de interrupción).

8 niveles de prioridad de **tipo b** (*IR0* la más alta, *IR7* la más baja). Se sirve la de mayor prioridad y se guardan las demás (sólo se guarda una por nivel).

Enmascaramiento individual para las prioridades de **tipo a**.

Tema 5 – Interrupciones

5.5. Controlador de Interrupciones Múltiples i8259



-*IRR (Interrupt Request Register):* almacena las señales de interrupción activas (cuasi-antirrebotes).
-*ISR (In-Service Register):* almacena las señales de interrupción que están siendo servidas (borrado).
-*IMR (Interrupt Mask Register):* almacena la máscara de interrupción de cada señal de interrupción (0→no inhibida, 1→inhibida) (enmascaramiento).
-El resolvedor de prioridades determina qué señal de interrupción tiene mayor prioridad.

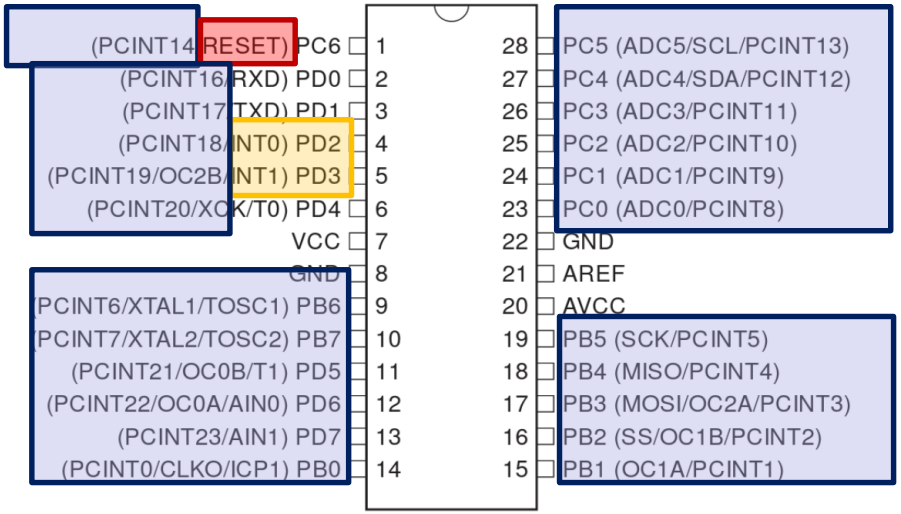
-La señal *INT* lleva la indicación de interrupción hacia la CPU. La señal *INTA* por parte de la CPU hace que el i8259 vuelque el vector de interrupción en el bus de datos.
-La línea *A0* selecciona registros. Las líneas *CAS0-2* portan un identificador de i8259 dentro de la cascada de i8259 conectados.

Tema 5 – Interrupciones

5.5. Interrupciones en el Atmega328p

Dispone de **26 interrupciones** vectorizadas con **mascaras individuales**:

- 2 señales de interrupción externa (por nivel o flanco) INT0-INT1
- 3 señales de interrupción externa (por flanco), PCI0-PCI1-PCI2, una por cada puerto digital que reciben señales de los pines PCINT0-23
- 20 señales de interrupción interna
- 1 señal de RESET



Tema 5 – Interrupciones

5.5. Interrupciones en el Atmega328p

VectorNo.	Program Address(2)	Source	Interrupt Definition
1	0x0000(1)	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time-out Interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1 COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1 COMPB	Timer/Coutner1 Compare Match B
14	0x001A	TIMER1 OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 Compare Match B
17	0x0020	TIMER0 OVF	Timer/Counter0 Overflow
18	0x0022	SPI, STC	SPI Serial Transfer Complete
19	0x0024	USART, RX	USART Rx Complete
20	0x0026	USART, UDRE	USART, Data Register Empty
21	0x0028	USART, TX	USART, Tx Complete
22	0x002A	ADC	ADC Conversion Complete
23	0x002C	EE READY	EEPROM Ready
24	0x002E	ANALOG COMP	Analog Comparator
25	0x0030	TWI	2-wire Serial Interface
26	0x0032	SPM READY	Store Program Memory Ready



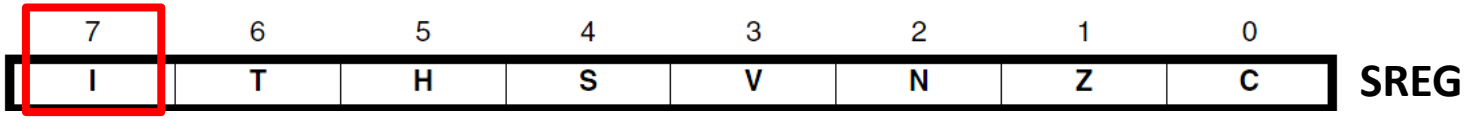
+ prioridad

- prioridad

Tema 5 – Interrupciones

5.5. Interrupciones en el Atmega328p

- **Global Enable**.- En el registro SREG se debe poner a 1 para que estén las interrupciones habilitadas (se usan las instrucciones CLI y SEI)



- **Máscaras individuales**. Cada interrupción está asociada a un bit que debe estar a 1 para que esté habilitada.
  - Por ejemplo para la conversion ADC (ADIE), para overflow del timer 1 (TOIE1):



Cuando se activa una interrupción, y su máscara está habilitada, la CPU deshabilita el I-bit, no guarda en la pila el estado, salta a la ISR, y retorna con RETI, que vuelve a habilitar el I-bit.

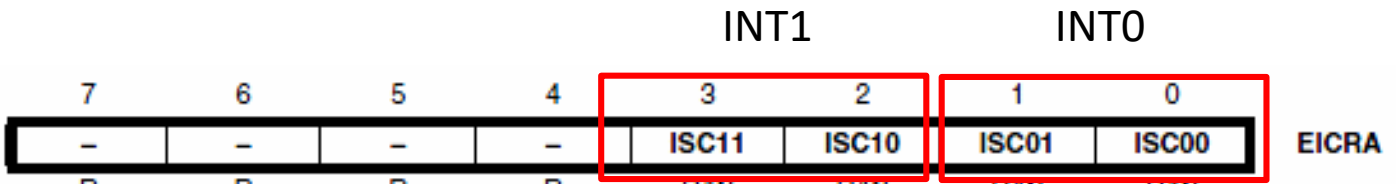
Tema 5 – Interrupciones

5.5. Interrupciones en el Atmega328p. External Interrupt

- Pines PD2 y PD3. Se deben configurar como entrada.

VectorNo.	Program Address(2)	Source	Interrupt Definition
1	0x0000(1)	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1

Se pueden configurar mediante el registro EICRA

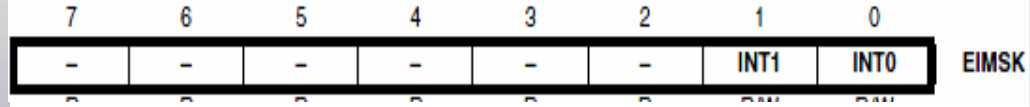


ISC01	ISC00	Description
0	0	The low level of INT0 generates an interrupt request.
0	1	Any logical change on INT0 generates an interrupt request.
1	0	The falling edge of INT0 generates an interrupt request.
1	1	The rising edge of INT0 generates an interrupt request.

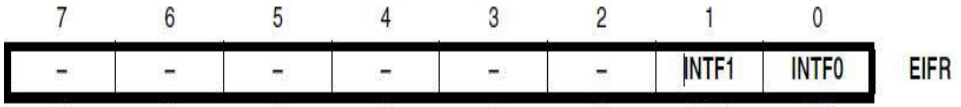
Tema 5 – Interrupciones

5.5. Interrupciones en el Atmega328p. External Interrupt

- **Registro de mascara (EIMSK)** para las interrupciones externas.



- **Registro de flags (EIFR)** de interrupciones externas. Se pondrán a 1 cuando esté activado I-Bit y la máscara individual correspondiente (opcional)




```
#include <avr/io.h>
#include <avr/interrupt.h>

ISR(INT0_vect)
{
    //toggle PB5
    PORTB^=(1<<5);
}
int main(void)
{
    //Desactivo interrupciones
    cli();
    // Port B output
    DDRB=0xFF;

    // Port D input (INT0 corresponde a PD2)
    DDRD=0x00;

    // Habilito mascara individual para INT0
    EIMSK=0x01;

    // Configuro activación a flanco de bajada
    EICRA=0x02;

    // Borro el Flag Register (opcional)
    EIFR=0x00;

    // Activo interrupciones
    sei();
    while(1);
}
```

Tema 5 – Interrupciones

5.5. Interrupciones en el Atmega328p. Pin Change

El Atmega328p dispone de la facilidad para generar interrupciones cuando se produce cualquier cambio en un pin de entrada. Es decir, dispone de 24 interrupciones adicionales en bloques (uno por cada puerto).

A diferencia de otras interrupciones que disparan una ISR individual, las interrupciones de cambio de PIN disparan una ISR para todo el puerto; en esa ISR habrá que detectar qué pin de entrada ha provocado la interrupción. Los registros para habilitarlas es el PCICR

PCICR – Pin Change Interrupt Control Register

Bit	7	6	5	4	3	2	1	0	
(0x68)	–	–	–	–	–	PCIE2	PCIE1	PCIE0	PCICR
Read/Write	R	R	R	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	



Tema 5 – Interrupciones

5.5. Interrupciones en el Atmega328p. Pin Change

Para enmascarar las interrupciones de cambio de pin haremos uso de los registros PCMSK0 (Puerto B), PCMSK1 (Puerto C) y PCMSK2 (Puerto D)

PCMSK0 – Pin Change Mask Register 0

Bit	7	6	5	4	3	2	1	0	
(0x6B)	PCINT7	PCINT6	PCINT5	PCINT4	PCINT3	PCINT2	PCINT1	PCINT0	PCMSK0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

PCMSK1 – Pin Change Mask Register 1

Bit	7	6	5	4	3	2	1	0	
(0x6C)	–	PCINT14	PCINT13	PCINT12	PCINT11	PCINT10	PCINT9	PCINT8	PCMSK1
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

PCMSK2 – Pin Change Mask Register 2

Bit	7	6	5	4	3	2	1	0	
(0x6D)	PCINT23	PCINT22	PCINT21	PCINT20	PCINT19	PCINT18	PCINT17	PCINT16	PCMSK2
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Tema 5 – Interrupciones

5.5. Interrupciones en el Atmega328p. Pin Change

Una sola ISR para todas las interrupciones del mismo puerto, alojadas en PCINT0_vect, PCINT1_vect y PCINT2_vect.

Ejemplo para el puerto B.

```
ISR(PCINT0_vect){

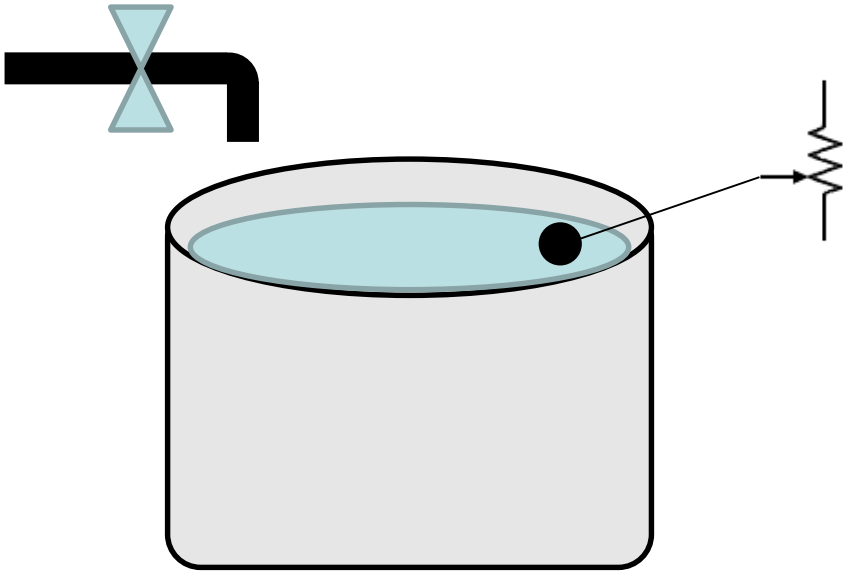
if ( PINB & (1<<PINB3)) {
          //Action for PCINT at PB3
}
else if (PINB & (1<<PINB4)){
          //Action for PCINT at PB4
}

}
```

Tema 5 – Interrupciones

Ejemplo:

Implantar un sistema de control del nivel de líquido de un tanque. El sistema comenzará cerrando la válvula de entrada, y a partir de ahí medirá cada 15 ms el nivel que señala un flotador (el cual da un voltaje analógico entre 0v y 5v, siendo 0v el tanque vacío y 5v el tanque lleno). Si el tanque está medio vacío (lectura < 2.5v) la válvula se abrirá. En caso contrario se cerrará. La apertura de la válvula es comandada por una onda PWM de 50Hz cuyo ancho positivo podrá valer linealmente desde 2mseg (válvula cerrada) hasta 10mseg (válvula abierta). El sistema puede habilitarse/deshabilitarse mediante un botón. En caso de que se deshabilite, se cerrará la válvula independientemente del nivel del flotador.



Tema 5 – Interrupciones

5.6. Resumen

- El uso de interrupciones resulta de vital importancia en sistemas de tiempo real, especialmente en sistemas de tiempo real real.
- Presentan algunas cuestiones que deben ser solventadas como rebotes en la generación.
- La gestión de múltiples interrupciones es también importante ya que un STR debe responder a múltiples eventos del exterior.
- Se debe considerar siempre la latencia de las interrupciones, así como el peor tiempo de ejecución de la rutina que la sirve (ISR)
- Se ha estudiado el manejador de interrupciones de Intel 8259